Requirement Analysis Document

DBMS Mini Project

1. Project Title

Campus Complaint & Grievance Tracker — A Trigger-Driven Complaint Lifecycle System

2. Introduction

The Campus Complaint & Grievance Tracker is a database-driven system designed for a university or college campus to record, track, and resolve complaints raised by students across every major area of campus life — hostel, mess, academics, infrastructure, lab, and library services. The system gives every complaint a traceable lifecycle from the moment it is filed to the moment it is resolved, instead of leaving it to informal channels.

Currently, complaints on most campuses are reported informally — through WhatsApp groups, verbal requests to wardens, or ad-hoc emails — with no structured record of what was reported, who is responsible, or whether it was ever addressed. The proposed system is required to replace this informal process with a centralized, rule-enforced database that tracks every complaint's status, owner, and history. It will be used by four types of users: Students (who file complaints), Staff (who resolve them), Wardens (who oversee hostel-level complaints and assign staff), and Admins (who oversee the system across all departments).

3. Problem Statement

Managing campus complaints manually or through informal messaging channels results in complaints going unrecorded, no visibility into whether an issue is being worked on, no accountability once a complaint is raised, and no escalation path when the responsible staff member does not act in time. There is also no consolidated record that lets the administration see which departments or categories generate the most complaints, or how long issues typically take to resolve. The proposed system aims to provide a centralized, database-driven system for managing students, staff, complaints, categories, departments, and their complete status history, with rules enforced at the database level rather than left to manual follow-up.

4. Objectives

1. To design a normalized relational database schema for a multi-department campus complaint system.
2. To develop a complaint lifecycle that is enforced as a database-level state machine, preventing invalid status jumps.
3. To implement automatic escalation of complaints that remain unresolved beyond their category's service level agreement (SLA).
4. To provide role-based views so Students, Staff, Wardens, and Admins each see only the data relevant to them.
5. To maintain a complete, automatically generated audit trail of every status change made to a complaint.
6. To optimize frequently used queries through indexing and to demonstrate measurable query performance improvement.

5. Functional Requirements

Functional requirements describe what the system should do.

FR No.	Functional Requirement
FR1	The system shall allow students to register and log in using valid credentials.
FR2	The system shall allow students to file a new complaint under a specific category.
FR3	The system shall allow students to view the status and full history of only their own complaints.
FR4	The system shall allow staff to view complaints currently assigned to them.
FR5	The system shall allow staff to update a complaint's status, subject to valid state-machine transitions.
FR6	The system shall allow staff to add resolution notes when closing a complaint.
FR7	The system shall allow wardens to view all complaints raised within their own hostel.
FR8	The system shall allow wardens to assign or reassign staff to a complaint.
FR9	The system shall allow admins to view complaints across every department.
FR10	The system shall automatically escalate a complaint if it remains unresolved beyond its category's SLA.
FR11	The system shall automatically log every status change into an audit trail, with no manual step required.
FR12	The system shall reject any invalid complaint status transition (e.g. Open directly to Resolved).
FR13	The system shall allow admins to generate reports on complaint volume, category trends, and average resolution time.
FR14	The system shall allow admins to manage department, category, and staff records.

6. User Roles and Permissions

Role	Permissions
Student	File a new complaint under any category; view the status and full history of only their own complaints; track escalation progress on those complaints.
Staff	View complaints assigned to them; update complaint status (subject to state-machine validation — no skipping steps); add resolution notes.
Warden	View all complaints raised within their own hostel; assign or reassign staff to a complaint; monitor escalation status hostel-wide.
Admin	View all complaints across every department; monitor average resolution time and category-wise trends; manage department, category, and staff records.

7. Major Modules

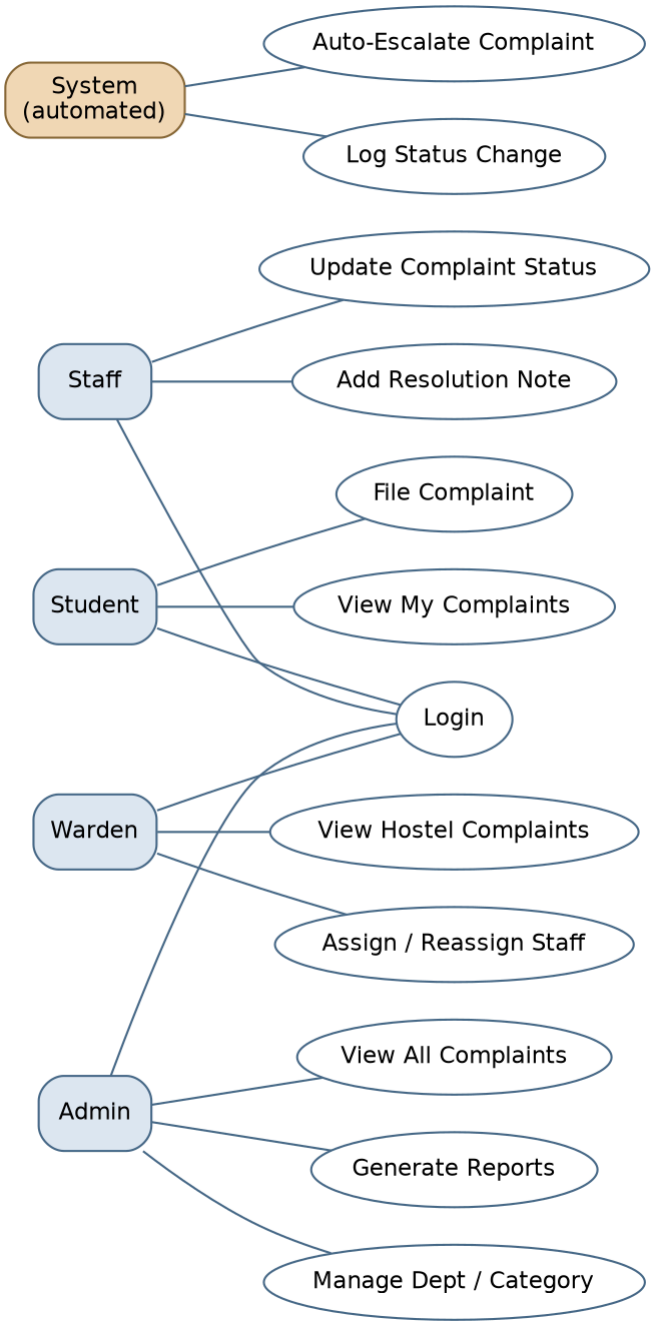
Module	Description
User Management	Handles registration, login, and role assignment for Students, Staff, Wardens, and Admins.
Complaint Management	Handles complaint creation, categorization, and linking each complaint to the student who filed it.

Module	Description
Status & Workflow Management	Enforces the complaint lifecycle (Open → Assigned → In-Progress → Resolved/Escalated) as a database-level state machine.
Escalation Management	Automatically escalates complaints that exceed their category's SLA, using a scheduled database event.
Assignment Management	Assigns and reassigns staff to complaints and keeps a history of all assignments.
Audit & History Management	Automatically logs every status change — old value, new value, who changed it, and when.
Report & Analytics Module	Generates category-wise trends, department performance, and average resolution time for admins.

8. Use Cases

Use Case ID	Use Case	Actor	Description
UC01	Login	Student / Staff / Warden / Admin	User logs into the system with valid credentials.
UC02	File Complaint	Student	Student submits a new complaint under a chosen category.
UC03	View My Complaints	Student	Student views the status and history of their own complaints.
UC04	Update Complaint Status	Staff	Staff moves a complaint to its next valid status.
UC05	Add Resolution Note	Staff	Staff records how a complaint was resolved.
UC06	View Hostel Complaints	Warden	Warden views all complaints from their hostel.
UC07	Assign / Reassign Staff	Warden	Warden assigns a staff member to handle a complaint.
UC08	View All Complaints	Admin	Admin views complaints across every department.
UC09	Generate Reports	Admin	Admin generates category-wise and resolution-time reports.
UC10	Manage Dept / Category	Admin	Admin creates or updates department and category records, including SLA.
UC11	Auto-Escalate Complaint	System (automated)	System escalates a complaint once it exceeds its category SLA.
UC12	Log Status Change	System (automated)	System records every status change into the audit trail automatically.

Use Case Diagram



9. Data Requirements

The system should not design the complete ERD here — that is developed in the next stage (Phase 2). This section only identifies the information the system needs to maintain.

Data Category	Data Items
User	User ID, Name, Email, Phone, Password, Role
Student	Roll Number, Hostel, Room Number

Data Category	Data Items
Staff	Designation, Department, Role Type (staff/warden/admin)
Department	Department ID, Department Name, Department Head
Category	Category ID, Category Name, SLA (days), Owning Department
Hostel	Hostel ID, Hostel Name, Warden
Complaint	Complaint ID, Filed By, Category, Description, Status, Created Date, Updated Date
Status History	History ID, Complaint ID, Old Status, New Status, Changed By, Changed At
Staff Assignment	Assignment ID, Complaint ID, Staff ID, Assigned At, Unassigned At

10. Business Rules

1. A student can file multiple complaints; each complaint must be filed by exactly one student.
2. Each complaint must belong to exactly one category, and each category belongs to exactly one department.
3. A complaint can only move through the sequence Open → Assigned → In-Progress → Resolved/Escalated — it cannot skip a step.
4. Each hostel has exactly one warden at any given time.
5. A staff member can be assigned to many complaints, but a complaint has exactly one actively responsible staff member at a time.
6. If a complaint remains unresolved beyond its category's SLA, it must be automatically escalated.
7. Every status change made to a complaint must be recorded in the status history — no change is allowed to go unlogged.
8. Only an Admin can delete or permanently remove department, category, or staff records.

11. Input Requirements

Input	Entered By	Purpose
User registration details	Admin	Create student and staff accounts
Login credentials	All users	Authentication
Complaint details (category, description)	Student	File a new complaint
Status update	Staff	Update complaint progress
Resolution notes	Staff	Document how a complaint was resolved
Staff assignment / reassignment	Warden	Assign the staff member responsible for a complaint
Department / Category / SLA details	Admin	Configure departments, categories, and SLA days

12. Output Requirements

Output / Report	Purpose
Complaint status view	Lets a student track their own complaint
Hostel-wide complaint list	Lets a warden monitor all complaints in their hostel
Admin complaint overview	Monitors all complaints across the institution
Average resolution time report	Measures department and staff performance
Category-wise complaint trend report	Identifies recurring problem areas
Escalated complaints report	Highlights overdue issues needing attention
Status history / audit log	Provides a complete accountability trail for any complaint

13. System Constraints and Assumptions

1. Users are assumed to have valid login credentials issued by the institution.
2. The system requires a centralized database server (MySQL or PostgreSQL).
3. Only authorized Admins can delete department, category, or staff records.
4. Internet connectivity is required to access the web-based frontend.
5. The system is designed for a single campus/institution, not a multi-tenant deployment.
6. Each student belongs to exactly one hostel at a time.
7. Escalation is evaluated on a daily scheduled basis rather than in real time.

14. Proposed Technology Stack

Component	Technology
Front-end	React (or HTML / CSS / JavaScript)
Back-end	Node.js / Express or Python / Flask
Database	MySQL / PostgreSQL
Programming Language	JavaScript / Python
Development Tools	VS Code, dbdiagram.io, Git / GitHub

15. Preliminary System Flow

User
↓
Login
↓
Role-Based Dashboard (Student / Staff / Warden / Admin)
↓
Select Action (File Complaint / Update Status / Assign Staff / View Reports)
↓
Database (Triggers, Views, Stored Procedures)
↓

Validate & Process (state-machine check, escalation check, logging)

↓

Display Result

16. Traceability of Requirements

Requirement	Module	Database Data
FR1	User Management	User
FR2, FR3	Complaint Management	Complaint, Category
FR4, FR5, FR6, FR12	Status & Workflow Management	Complaint, StatusHistory
FR7, FR8	Assignment Management	Staff_Assignment, Hostel
FR9, FR13	Report & Analytics Module	Complaint, Department, Category
FR10	Escalation Management	Complaint, Category (sla_days)
FR11	Audit & History Management	StatusHistory
FR14	User Management	Department, Category, Staff

17. Summary

The Campus Complaint & Grievance Tracker replaces the current informal, untracked way campus complaints are reported with a centralized, database-driven system. By enforcing the complaint lifecycle as a database-level state machine, automatically escalating overdue complaints based on category-specific SLAs, and maintaining a complete audit trail, the system directly addresses the lack of accountability, visibility, and escalation identified in the problem statement — while giving administrators the reporting data needed to identify recurring issues and measure department performance.

18. References

1. Elmasri, R., & Navathe, S. B. (2018). Fundamentals of Database Systems (7th ed.). Pearson Education.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2013). Database System Concepts (7th ed.). McGraw Hill Education.
3. MySQL 8.0 Reference Manual — https://dev.mysql.com/doc/
4. PostgreSQL Documentation — https://www.postgresql.org/docs/
5. dbdiagram.io Documentation — https://dbdiagram.io/docs